

Name : Anmol Sharma

Section : 2A

University Roll No. : 2415500076

Class Roll No. : 10

Subject : Probability and Statistics

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
plt.figure(figsize=(10,10))
```

```
<Figure size 1000x1000 with 0 Axes>
<Figure size 1000x1000 with 0 Axes>
```

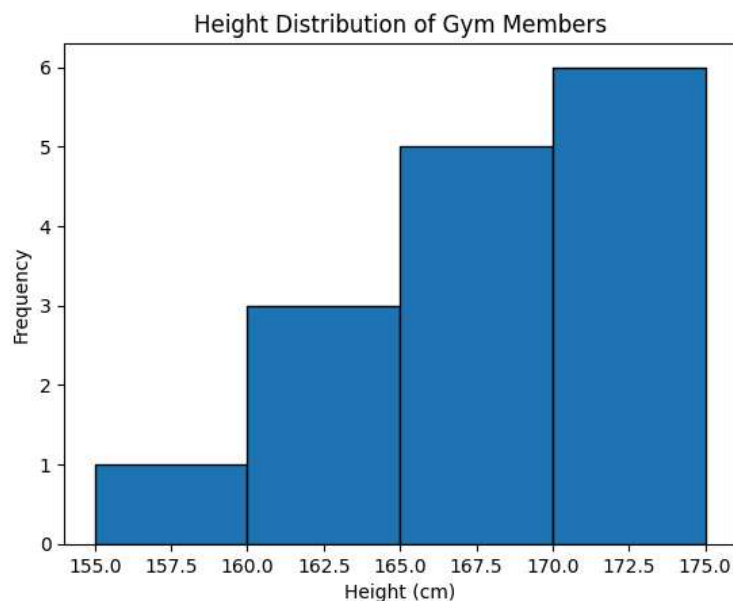
Task 1 : Task 1: The Gym Client – “Heights Matter” A fitness trainer has collected the heights of 15 gym members:

162, 158, 165, 170, 172, 168, 175, 160, 166, 171, 167, 169, 173, 174, 161

The trainer wants to know:

- How the heights are distributed in groups.
- A clear visual histogram to show his clients. Use your skills to create a grouped frequency distribution and a histogram.

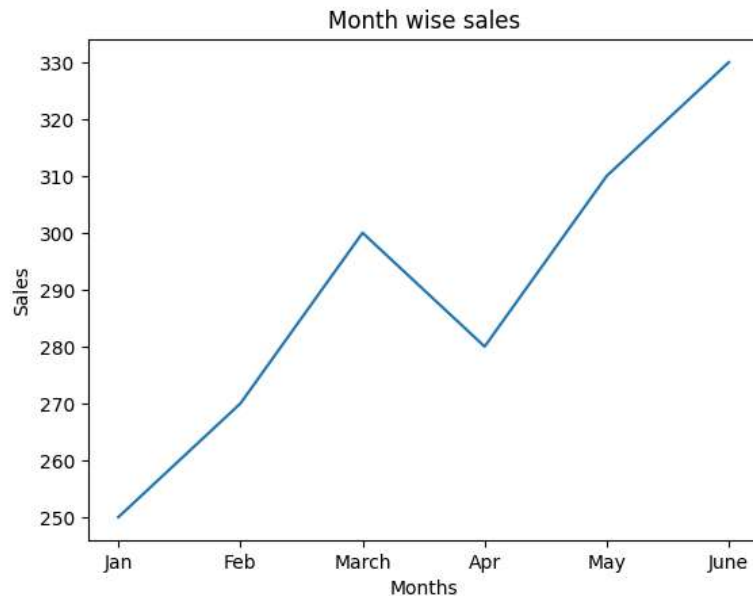
```
heights = [162,158,165,170,172,168,175,160,166,171,167,169,173,174,161]
bins = np.arange(155, 180, 5)
plt.hist(heights, bins=bins, edgecolor='black')
plt.title("Height Distribution of Gym Members")
plt.xlabel("Height (cm)")
plt.ylabel("Frequency")
plt.show()
```



Task 2 : A shopkeeper, Meera, recorded sales for 6 months: Jan=250, Feb=270, Mar=300, Apr=280, May=310, Jun=330 She is preparing for an investor meeting and says: “I don’t want boring tables, I want a story of growth.” Plot a line graph to visualize her sales trend.

```
month=["Jan", "Feb", "March", "Apr", "May", "June"]
sales=[250,270,300,280,310,330]
plt.plot(month,sales)
plt.xlabel("Months")
```

```
plt.ylabel("Sales")
plt.title("Month wise sales")
plt.show()
```



Task 3 : A teacher tracked students' study hours and scores:

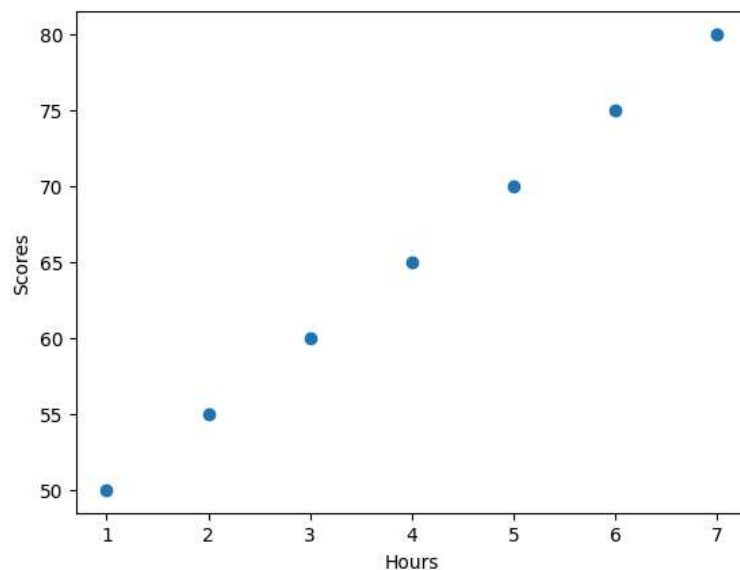
Study Hours 1 2 3 4 5 6 7

Scores 50 55 60 65 70 75 80

She wants to prove that more study hours improve marks. Create a scatter plot to show the relationship.

```
hours=[1,2,3,4,5,6,7]
scores=[50,55,60,65,70,75,80]
plt.scatter(hours,scores)
plt.xlabel("Hours")
plt.ylabel("Scores")
```

Text(0, 0.5, 'Scores')



Task 4 : A shopping mall hired you to study customer behavior using their dataset (downloaded from Kaggle). You must:

- Load the dataset and display the first five rows.
- Print the column names and data types.
- Find the number of rows and columns.



(d) Identify each variable as Nominal, Ordinal, Interval, or Ratio.

(e) Visualize by:

- A histogram of Age
- A scatter plot between Annual Income and Spending Score The manager will use this to design targeted offers.

```
data = pd.read_csv("/content/Customer_Behaviour.csv")
```

```
data.head(5)
```

	User ID	Gender	Age	EstimatedSalary	Purchased	
0	15624510	Male	19	19000	0	
1	15810944	Male	35	20000	0	
2	15668575	Female	26	43000	0	
3	15603246	Female	27	57000	0	
4	15804002	Male	19	76000	0	

Next steps: [Generate code with data](#) [New interactive sheet](#)

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   User ID               400 non-null   int64
1   Gender                400 non-null   object
2   Age                   400 non-null   int64
3   EstimatedSalary       400 non-null   int64
4   Purchased             400 non-null   int64
dtypes: int64(4), object(1)
memory usage: 15.8+ KB
```

```
data.count()
```

```

      0
User ID    400
Gender     400
Age        400
EstimatedSalary  400
Purchased  400

dtype: int64
```

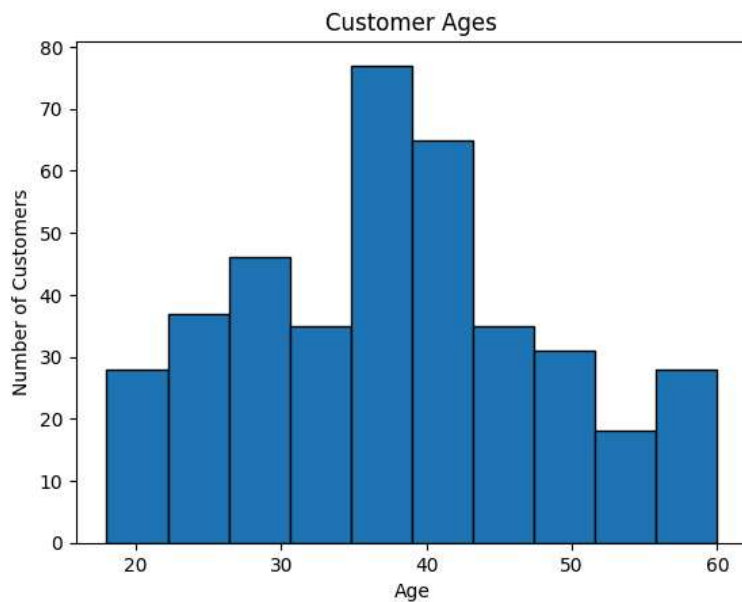
Nominal : Gender

Ratio : Age , Estimated Salary

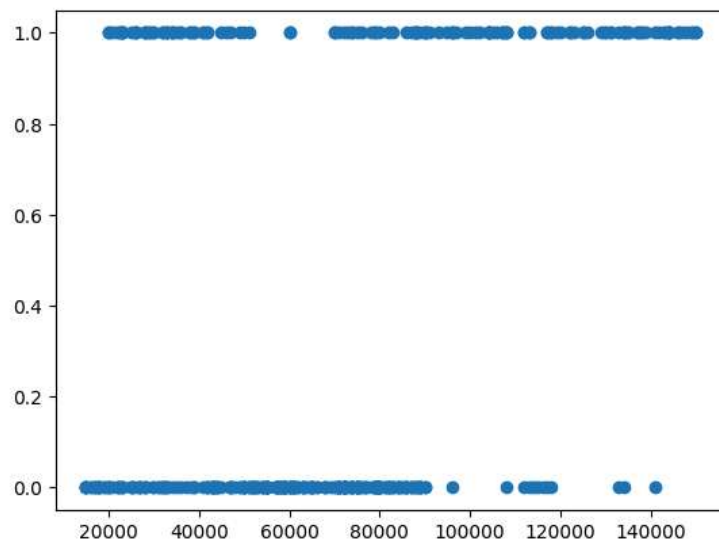
Ordinal : UserID

```
plt.hist(data['Age'],edgecolor='black')
plt.xlabel("Age")
plt.ylabel("Number of Customers")
plt.title("Customer Ages")
plt.show()
```





```
plt.scatter(data['EstimatedSalary'], data['Purchased'])
plt.show()
```



Task 5 : A startup is creating a new online game. The first round is a coin toss twice. Simulate the two tosses and list all possible outcomes.

```
import itertools

outcomes = list(itertools.product(['H','T'], repeat=2))
print("All possible outcomes:", outcomes)
```

All possible outcomes: [('H', 'H'), ('H', 'T'), ('T', 'H'), ('T', 'T')]

Task 6 : A café hosting board games wants a fair dice guide. Simulate rolling a die and calculate its Probability Mass Function (PMF).

```
import numpy as np

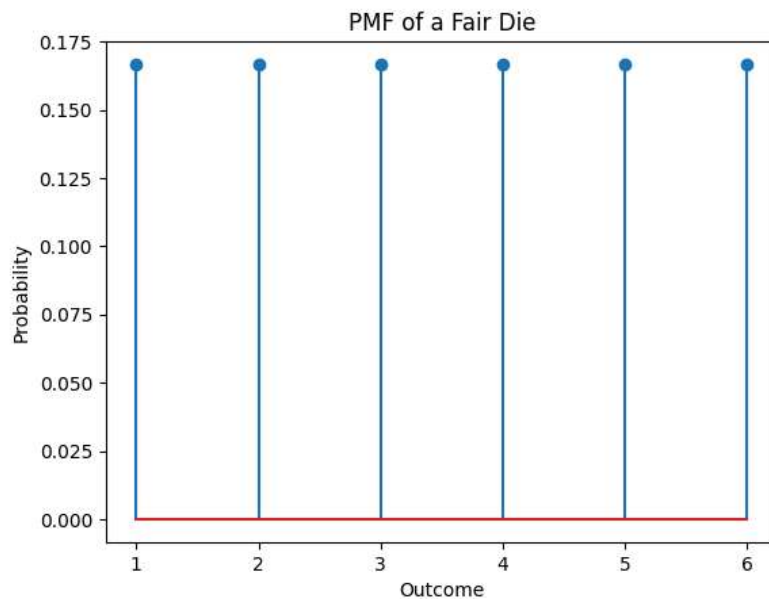
rolls = np.arange(1,7)
pmf = [1/6]*6
for r,p in zip(rolls,pmf):
    print(f"P(X={r}) = {p:.2f}")

plt.stem(rolls, pmf)
plt.title("PMF of a Fair Die")
```



```
plt.xlabel("Outcome")
plt.ylabel("Probability")
plt.show()
```

```
P(X=1) = 0.17
P(X=2) = 0.17
P(X=3) = 0.17
P(X=4) = 0.17
P(X=5) = 0.17
P(X=6) = 0.17
```



Task 7 : In a screw manufacturing factory, the weights of screws follow a normal distribution. Generate a dataset of screw weights and plot both:

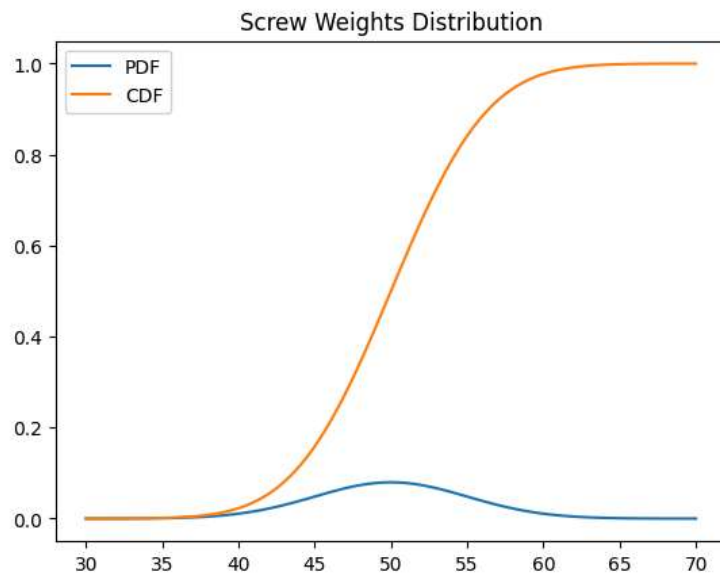
- The PDF (Probability Density Function)
- The CDF (Cumulative Distribution Function)

```
from scipy.stats import norm

weights = np.random.normal(50, 5, 1000)
x = np.linspace(30, 70, 100)
pdf = norm.pdf(x, 50, 5)
cdf = norm.cdf(x, 50, 5)

plt.plot(x, pdf, label='PDF')
plt.plot(x, cdf, label='CDF')
plt.legend()
plt.title("Screw Weights Distribution")
plt.show()
```





Task 8 : A production manager reports the following distribution of defects per batch:

Defects (X) 0 1 2 3

$P(X)$ 0.1 0.3 0.4 0.2

Tasks:

(a) Check if this is a valid PMF.

(b) Compute the cumulative probabilities and plot the CDF.

```
import numpy as np

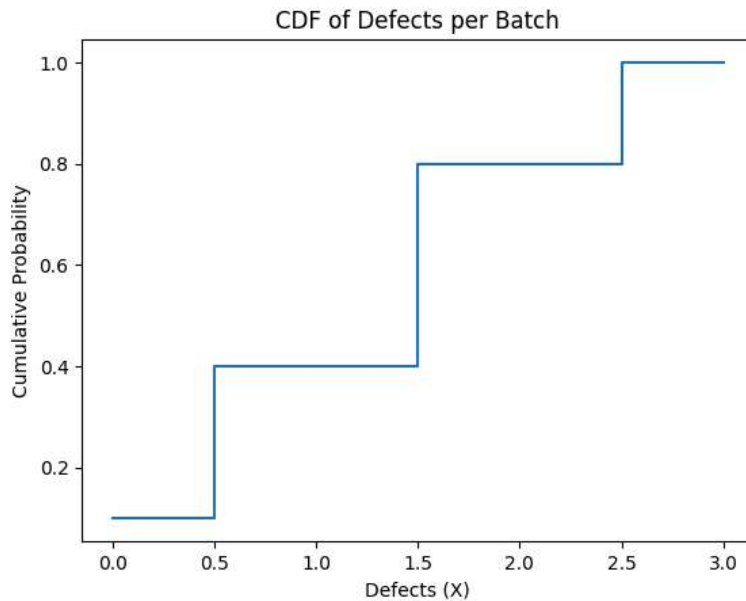
x = np.array([0,1,2,3])
p = np.array([0.1,0.3,0.4,0.2])

print("Valid PMF?", np.isclose(p.sum(),1))

cdf = np.cumsum(p)
plt.step(x, cdf, where='mid')
plt.title("CDF of Defects per Batch")
plt.xlabel("Defects (X)")
plt.ylabel("Cumulative Probability")
plt.show()
```



Valid PMF? True



Task 9 : A call center receives different numbers of support tickets daily. Explain and model:

- Sample space and events
- Discrete and continuous random variables
- PMF, PDF, and CDF for tickets

Ans:-

Sample Space: {0,1,2,3,...}

Event: e.g., number of tickets > 5

Discrete Variable: Number of tickets

Continuous Variable: Time between calls

PMF: Probability of exact ticket count

PDF: Probability density for continuous times

CDF: Cumulative probability up to a point

Task 10 : The meteorological department tracks daily rain/no-rain for 30 days. Simulate daily weather and present:

- The PMF (frequency of rainy vs. dry days)
- The CDF (cumulative rainy days over 30 days)

```
days = np.random.choice(['Rainy','Dry'], size=30, p=[0.4,0.6])
pmf = pd.Series(days).value_counts(normalize=True)
cdf = pmf.cumsum()

print("PMF:\n", pmf)
print("\nCdf:\n", cdf)

pmf.plot(kind='bar', color=['blue','orange'], title='PMF of Rainy vs Dry Days')
plt.show()

cdf.plot(kind='bar', color='green', title='CDF of Weather')
plt.show()
```



```
PMF:  
Dry      0.633333  
Rainy    0.366667  
Name: proportion, dtype: float64
```

```
CDF:  
Dry      0.633333  
Rainy    1.000000  
Name: proportion, dtype: float64
```

